

ApexPlanet Cybersecurity Internship - Task 5

FullReport

NAME : SHIVAM KUNAL

EMAIL.ID : shivamkunal98@gmail.com

Intern : Cybersecurity & Ethical Hacking

- **Task 5 : Capstone Project & Incident Response**

Objective: Apply all skills in a self-chosen capstone project and simulate an incident response.

Project Title

Web Application Penetration Testing Report on DVWA (Damn Vulnerable Web Application)

Objective

The objective of this project was to perform a comprehensive security assessment of DVWA in a controlled laboratory environment. The assessment focused on identifying common web application vulnerabilities, evaluating their impact, and recommending appropriate mitigation measures.

Scope

- Target: DVWA
- Environment: Kali Linux & DVWA Lab
- Testing Type: Black Box and Manual Security Testing

-
- Focus Areas:
 - SQL Injection
 - Cross-Site Scripting (XSS)
 - Cross-Site Request Forgery (CSRF)
 - File Inclusion Vulnerabilities
 - Security Misconfigurations

2. Project Planning

Tools Used

Offensive Security Tools

- Burp Suite
- SQLMap
- Firefox Developer Tools
- cURL

Analysis Tools

- Wireshark
- SecurityHeaders

Environment

- Kali Linux
- Apache Server
- DVWA

3. Methodology

The penetration testing process followed the industry-standard methodology:

Penetration Testing Methodology

Overview

The penetration testing assessment was conducted using a structured methodology based on industry-standard security testing practices. The objective was to identify vulnerabilities, evaluate their impact, and recommend mitigation strategies for improving the security posture of the web application.

The assessment followed five major phases:

1. Information Gathering
 2. Vulnerability Assessment
 3. Exploitation
 4. Post-Exploitation Analysis
 5. Reporting & Remediation
-

Phase 1: Information Gathering (Reconnaissance)

Objective

The purpose of this phase was to understand the application's structure, functionality, and attack surface before performing security testing.

Activities Performed

Application Mapping

The entire DVWA application was manually explored to identify:

- Login functionality
- User input fields
- URL parameters
- Forms and buttons
- Authentication mechanisms
- Session handling processes

Parameter Enumeration

The application URLs were analyzed to identify dynamic parameters that interact with backend databases.

Examples:

`http://localhost/dvwa/vulnerabilities/sqli/?id=1`

Dynamic parameters often become targets for injection-based attacks.

Session Analysis

Cookies and session identifiers were inspected using Burp Suite to understand:

- Session management
- Authentication handling
- Cookie security attributes

Technology Fingerprinting

The underlying technologies were identified:

- Apache Web Server
- PHP
- MySQL Database

Outcome

This phase provided a complete understanding of the application's attack surface and identified potential entry points for further testing.

Phase 2: Vulnerability Assessment

Objective

The goal of this phase was to discover security weaknesses within the application.

Testing Approach

A combination of:

- Manual Testing
- Automated Testing
- Browser Inspection
- Proxy-Based Analysis

was used.

SQL Injection Testing

Procedure

Input fields were tested with special characters and database-related payloads to determine whether user input was properly validated.

Indicators tested:

- Error messages
- Unexpected database responses
- Authentication bypass possibilities

Purpose

To determine whether attackers could manipulate backend SQL queries.

Cross-Site Scripting (XSS) Testing

Stored XSS Testing

User-controllable input fields were examined to determine whether malicious scripts could be permanently stored in the application.

Reflected XSS Testing

URL parameters and search fields were tested to determine whether input was reflected back without sanitization.

Purpose

To identify opportunities for client-side script execution.

CSRF Testing

Procedure

Sensitive actions such as password changes were analyzed to determine whether requests could be executed without user consent.

Areas Examined

- Password change functionality
- Account modification features
- Session validation controls

Purpose

To determine whether unauthorized requests could be performed on behalf of authenticated users.

File Inclusion Testing

Local File Inclusion (LFI)

Input parameters associated with file loading functions were examined.

Remote File Inclusion (RFI)

Application functionality was reviewed for unsafe inclusion of external resources.

Purpose

To determine whether attackers could access sensitive files or execute unintended content.

Phase 3: Exploitation

Objective

The objective of this phase was to safely demonstrate the impact of identified vulnerabilities in a controlled environment.

SQL Injection Exploitation

The vulnerable parameter was manipulated to demonstrate unauthorized database access.

The following were assessed:

- Data disclosure
- Database enumeration
- Authentication weaknesses

Impact Demonstrated

- Exposure of user records
 - Access to backend information
 - Disclosure of sensitive data
-

XSS Exploitation

Stored and reflected XSS vulnerabilities were demonstrated to show how malicious scripts could execute within users' browsers.

Impact Demonstrated

- Session theft
 - User impersonation
 - Credential harvesting scenarios
-

CSRF Exploitation

A proof-of-concept request was developed to demonstrate unauthorized account modification actions.

Impact Demonstrated

- Password modification
 - Unauthorized actions under a victim account
-

File Inclusion Exploitation

File inclusion vulnerabilities were tested to demonstrate access to sensitive files.

Impact Demonstrated

- Configuration exposure
 - Sensitive information disclosure
-

Phase 4: Post-Exploitation Analysis

Objective

After successfully validating vulnerabilities, the potential business and security impacts were analyzed.

Activities Performed

Risk Analysis

Each vulnerability was classified according to:

- Confidentiality Impact
- Integrity Impact
- Availability Impact

Severity Assessment

Severity levels were assigned:

Severity	Description
Critical	Immediate risk to application security
High	Significant security impact
Medium	Moderate impact
Low	Limited impact

Attack Chain Evaluation

Potential attack paths were analyzed to determine how multiple vulnerabilities could be chained together by an attacker.

Example:

SQL Injection
↓

Credential Extraction

↓

Account Compromise

↓

Privilege Escalation

Phase 5: Reporting & Remediation

Objective

The final phase focused on documenting findings and recommending corrective actions.

Documentation Process

For every vulnerability the following information was recorded:

Vulnerability Name

Example:

SQL Injection

Description

Detailed explanation of the vulnerability.

Risk Level

Critical / High / Medium / Low

Evidence

Screenshots and proof-of-concept demonstrations.

Impact

Potential consequences if exploited.

Remediation

Recommended security controls and mitigation techniques.

Testing Limitations

The assessment was conducted within a controlled DVWA laboratory environment.

The purpose of testing was educational and focused on understanding common web application vulnerabilities. No unauthorized systems were targeted, and all testing activities were performed ethically and within the defined scope.

4. Vulnerability Analysis

Vulnerability 1: SQL Injection

Description

SQL Injection occurs when user input is improperly validated and executed directly within database queries.

Testing Procedure

1. Navigate to SQL Injection module.
2. Enter test payloads.
3. Observe database responses.
4. Extract database information.

Risk Level

Critical

Impact

- Unauthorized database access
- Information disclosure
- Authentication bypass

Evidence

(Add screenshots here)

Mitigation

- Prepared Statements
- Parameterized Queries
- Input Validation
- Least Privilege Database Accounts

Vulnerability 2: Stored XSS

Description

Stored XSS allows malicious scripts to be permanently stored and executed when users access affected pages.

Testing Procedure

1. Submit test payload in comment field.
2. Save entry.
3. Revisit page.
4. Observe script execution.

Risk Level

High

Impact

- Session Hijacking
- Cookie Theft
- User Impersonation

Evidence

(Add screenshots)

Mitigation

- Output Encoding
 - Input Validation
 - Content Security Policy (CSP)
-

Vulnerability 3: Reflected XSS

Description

Reflected XSS occurs when user input is immediately reflected back without sanitization.

Risk Level

Medium

Impact

- Phishing
- Session Theft
- Client-side Attacks

Mitigation

- Input Sanitization
 - Output Encoding
 - CSP Headers
-

Vulnerability 4: CSRF

Description

Cross-Site Request Forgery tricks authenticated users into performing unintended actions.

Testing Procedure

1. Analyze password change functionality.
2. Create malicious request.
3. Execute request.

Risk Level

High

Impact

- Unauthorized Account Changes
- User Manipulation

Mitigation

- CSRF Tokens
- SameSite Cookies
- User Re-authentication

Vulnerability 5: Local File Inclusion (LFI)

Description

Allows attackers to access sensitive local files.

Risk Level

High

Impact

- Configuration Disclosure
- Credential Exposure

Mitigation

- Input Validation
 - File Whitelisting
 - Restrict File Access
-

5. Security Headers Assessment

Tested Headers

Content-Security-Policy

Purpose:

Prevents execution of unauthorized scripts.

X-Frame-Options

Purpose:
Protects against clickjacking attacks.

X-Content-Type-Options

Purpose:
Prevents MIME type sniffing.

Strict-Transport-Security

Purpose:
Forces HTTPS communication.

Recommendation

Implement all critical security headers for enhanced application security.

6. Risk Assessment Table

Vulnerability	Severity	Status
SQL Injection	Critical	Found
Stored XSS	High	Found
Reflected XSS	Medium	Found
CSRF	High	Found
LFI	High	Found

7. Incident Response Simulation

Scenario

An attacker successfully exploited SQL Injection and gained access to sensitive user information.

Detection

- Suspicious database queries detected.
- Unusual application logs observed.

Containment

-
- Disable vulnerable page.
 - Block suspicious IP addresses.

Eradication

- Patch vulnerable code.
- Validate all user inputs.

Recovery

- Restore secure application version.
- Verify database integrity.

Lessons Learned

- Security testing should be performed regularly.
 - Secure coding practices must be implemented.
 - Continuous monitoring is essential.
-

8. Recommendations

Application Security

- Input Validation
- Output Encoding
- Prepared Statements

Authentication Security

- Strong Password Policies
- Multi-Factor Authentication

Infrastructure Security

- HTTPS Enforcement
 - Security Headers
 - Regular Vulnerability Assessments
-

9. Conclusion

The penetration test successfully identified multiple web application vulnerabilities within the DVWA environment. The findings demonstrate the importance of secure coding practices, regular security assessments, and proactive vulnerability management. Implementing the recommended mitigations would significantly improve the application's security posture.
